

Writing R Packages

Rohit Goswami · 2023-11-10 · r · cpp · bindings · packaging

Some thoughts on melding R with lower level languages and package management

Background

I like plotting with `ggplot2`. It makes more sense to me than the other plotting systems. These notes arose while binding `readCon` and `potlib` for `cuh2vizR`. However, for best practices, my `fastMatMR` project should be consulted.

Idiosyncracies

- I find it odd that R packages commit so much generated code
 - Can be simplified with a CI bot

Repo setup

`pixi` with `renv` turns out to be surprisingly reliable. I normally begin with:

```
echo ".pixi/" > .gitignore
gibo dump R C++ | cat >> .gitignore
pixi init
pixi add radian r-renv compilers
git add .
git commit -m "PRJ: Let there be light"
```

The rest of the preliminaries can take place within the context of `renv` and `pixi shell`. I tend to leave `LICENSE` and other templates to `usethis`.

renv package development

This is fairly straightforward, but takes a while to run, from the `radian` terminal (after `pixi-shell`).

```
renv::init()
install.packages(c("devtools")) # can take a while
usethis::create_package(".") # from $GITROOT
```

Now we can start with the rest of the templates.

```
usethis::use_mit_license()
usethis::use_readme_rmd()
```

****Warning****

CRAN doesn't like fixed dependencies, see [the discussion here](https://github.com/ropensci-review-tools/pkgcheck/issues/149), and often for developing packages one might not need `renv` though it still makes sense if one has multiple development machines. This turned out to be something I regretted, though `pixi` is still useful.

Stylistic decisions

Always a good idea to lint and style from the get-go, so:

```
install.packages(c("lintr", "styler"))
```

Also remember to add `.pixi/` to `.Rbuildignore` otherwise local checks will parse the entire contents of the rather substantial folder.

Pre-commit hooks

For my packages, I tend to use a `pre-commit-hook` configured with `.pre-commit-config.yaml`:

```
repos:
- repo: https://github.com/pre-commit/pre-commit-hooks
  rev: v4.4.0
  hooks:
  - id: trailing-whitespace
    exclude: ^(.lintr)
  - id: end-of-file-fixer
  - id: check-yaml
  - id: check-added-large-files
- repo: https://github.com/pre-commit/mirrors-clang-format
  rev: v16.0.6
  hooks:
  - id: clang-format
```

Which can be used via:

```
# Check
pixx run pre-commit run -a
# Install
pixx run pre-commit install
```

Also worth noting is the CI configuration for the same especially useful for contributors,

`.github/workflows/pre-commit.yml`:

```
name: pre-commit
on:
  pull_request:
  push:
    branches: [main]
jobs:
  pre-commit:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - uses: actions/setup-python@v3
        with:
          python-version: '3.11'
      - uses: pre-commit/action@v2.0.3
```

However, sometimes `pre-commit` for `R` can be flakey. In such situations consider:

```
find . \( -path "./.pixi" -o -path "./renv" \) -prune -o -type f -name "*.R" -exec Rscript -e
'library(styler); style_file("{}")' \;
Rscript -e 'library(lintr); lintr::lint_package(".")'
```

In any case, always remember to run `renv::snapshot()` after an `install.packages` call.

ROpenSci package check

Warning

This is **really** slow locally, but required in the `renv` workflow. Still annoying to run locally!

We need to install this with a non-CRAN [source](#):

```
options (repos = c (
  ropensci-reviewtools = "https://ropensci-review-tools.r-universe.dev",
  CRAN = "https://cloud.r-project.org"
))
install.packages("pkgcheck")
```

When its finally done, add the CI configuration.

```
name: ROpenSci pkg-check
on:
  push:
    branches:
      - main
  pull_request:
    branches:
      - main
jobs:
  pkgcheck:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
        with:
          submodules: "recursive"
          fetch-depth: 0
      - name: pkgcheck
        uses: ropensci-review-tools/pkgcheck-action@v1.0.0
```

Using `cpp11`

Even after `usethis::use_package("cpp11")` one must link to it explicitly in the `DESCRIPTION`:

```
SystemRequirements: C++17
Encoding: UTF-8
Roxygen: list(markdown = TRUE)
RoxygenNote: 7.2.3
+LinkingTo:
+  cpp11
Imports:
  cpp11
```

One of the other things which tripped me up is that the functions are not automatically exported, and so one needs to create, in `R/$PKGNAME-package.R`:

```
## usethis namespace: start
#' @useDynLib your-package-name, .registration = TRUE
## usethis namespace: end
#' @export func-name-from-cpp11.R
NULL
```

Finally, in most cases I found `devtools::load_all()` to be a bit insufficient, had to use the more complete:

```
devtools::clean_dll()
cpp11::cpp_register()
devtools::document()
devtools::load_all()
```

I never understood why the `R` community commits so many generated files, personally I like to prepend my `gitignore` with:

```
## Generated files
R/cpp11.R
src/cpp11.cpp
man/
```

****Warning****

Sadly the entire `R`` ecosystem of checks will fail with this, so a first approximation committing everything might make some sense..

Later, a bot or CI action can be used to populate a “full” branch of the repository without muddying my nice `git` history. In fact, some of these are actually already part of the set of [Github Actions](#) supported by `usethis::`. I ended up with:

```
usethis::use_github_action() ## Basic, smoke test
usethis::use_github_action("check-standard") ## Overwrite older
```

I was also initially thrown by the requirement of including copies of source code. I suspect this might be bypassed by using `meson` as I have in `cuh2vizR`, unless CRAN packages are built without internet access. It would make it much easier to handle updates if `meson` were to handle setting up the dependencies, however, it isn't exactly part of `R-tools` [^fn:1].

Appeasing pkgcheck

Some thoughts:

- No `renv` in `.Rprofile`
- Use `codemeta` to get a `codemeta.json`
- Add a `CONTRIBUTING.md` (e.g. [here](#))
 - Not strictly necessary but I also go with a `CODE_OF_CONDUCT.md` (e.g. [here](#))

Personally I needed some `tex` updates:

```
tlmgr install collection-latexrecommended
tlmgr install collection-fontsrecommended
```

Always best to include a `.covrignore` but when in a pinch, this will work:

```
covr::package_coverage(line_exclusions=file.path(".pixi", list.files(path=".pixi", recursive=T)))
```

Also useful was:

```
usethis::use_build_ignore(c("newsfragments/", "pixi.*", "CONTRIBUTING.md", "CODE_OF_CONDUCT.md", ".covrignore"))
```

It is also important, for benchmarking and other long running vignettes, to have them cached or precomputed, as detailed in [this blog post](#) which in turn references an [older ROpenSci post](#). Vignette images are a whole other beast, nicely [covered here](#).

Conclusions

These were my rough notes while developing guidelines during the first release of [fastMatMR](#). There were additional changes to my packages, mostly in terms of making them more in keeping with the [R](#) community expectations and best practices. I went through the [ROpenSci](#) process and eventually also released to [CRAN](#) , but will cover that in a follow-up.